



Relatório de Auditoria de Segurança — JRC WhatsApp Broker

Fase 1

Incremento 2 — Console web operacional JRC

Referência da fonte

sha256:91f865cd22db3301885098a077d4dd8b1723290a683eda3bd9d05193e81e7cf9

Data de referência 2026-09-06

Resumo executivo

Foram consolidados 1 achado(s), 13 ponto(s) forte(s) e 18 rotas públicas. O gate bloqueia exclusivamente achados CRITICAL ou HIGH em estado OPEN.

1

Achados

0

Bloqueantes

13

Pontos fortes

Escopo e nota metodológica

Escopo: console web operacional, sessão de navegador, backend multicliente, PostgreSQL/RLS, autenticação, RBAC, API keys, instâncias, providers, OpenAPI, dependências, histórico Git JRC, bundles compilados e container. Permanecem fora: Meta real, mensageria, mídia, webhooks completos, billing, WhatsApp Calling, chatbot, filas e deploy.

Método: evidência em código e linhas, testes automatizados, inventário derivado do OpenAPI e scanners com saída sanitizada. O submódulo Evolution não é varrido historicamente; seu pin e sua fronteira são validados separadamente.

Eixos obrigatórios: isolamento multitenant; autorização não dependente somente do frontend; IDOR; exposição de segredos; inputs inseguros/XSS. Todos são aplicáveis neste incremento.

Categorias: CRITICAL — comprometimento imediato (P1); HIGH — impacto grave (P2); MEDIUM — impacto relevante condicionado (P3); LOW — hardening ou impacto limitado (P4); STRENGTH — controle positivo comprovado, sem efeito bloqueante.

CRITICAL

HIGH

MEDIUM

LOW

STRENGTH

Distribuição por severidade



Rosca por severidade

A rosca exclui INFO, reservado a contexto e não aplicabilidade. Ausência de segmentos indica zero achados CRITICAL, HIGH, MEDIUM ou LOW.

Evidências por categoria

AUTHENTICATION		1
AUTHORIZATION_RBAC		1
CONTAINER_RUNTIME		1
IDEMPOTENCY_DOS		1
IDOR		1
INPUTS_XSS		2
LOGGING_AUDIT		1
MULTITENANT_ISOLATION		1
OPENAPI_ROUTES		1
PROVIDER_BOUNDARY		1
RATE_LIMITING		1
SECRETS_CRYPTOGRAPHY		1
SUPPLY_CHAIN		1

Pontos fortes

JRC-STRENGTH-201

Troca de tenant invalida requests e estado local

O cliente incrementa a geração, aborta requests e executa os purges registrados antes de publicar a organização seguinte.

JRC-STRENGTH-202

Permissões do frontend são somente UX e o servidor permanece autoritativo

O navegador oculta ações incompatíveis com o papel, enquanto o guard Fastify revalida cada permissão no backend.

JRC-STRENGTH-203

Provider accounts e instâncias são resolvidos no tenant ativo

A organização vem da autenticação, a operação usa transação tenant e RLS, e identificadores de outro tenant não são revelados.

JRC-STRENGTH-204

Refresh HttpOnly e segredos voláteis no navegador

O refresh usa cookie host-only; access token fica em closure; QR, pairing e API key emitida são eliminados do estado volátil.

JRC-STRENGTH-205

Conteúdo não confiável não vira HTML ou navegação

React mantém escaping padrão e o QR é aceito somente como PNG base64 estrito, sem URL arbitrária nem dangerouslySetInnerHTML.

JRC-STRENGTH-206

Sessão de navegador valida Origin, CSRF e rotação

Rotas cookie-auth exigem origem exata e CSRF assinado; restore e switch rotacionam refresh com uso único.

JRC-STRENGTH-207

Rate limit distribuído persiste somente chaves HMAC

IP canônico e identidade normalizada são derivados com segredos distintos antes de entrar no Redis.

JRC-STRENGTH-208

Logger e E2E rejeitam canários sensíveis

A redaction estrutural cobre headers, cookies e desafios; o teardown falha se credenciais, QR, pairing ou identidade sintética chegarem aos logs.

JRC-STRENGTH-209

Evolution permanece atrás de contratos JRC

O registry separa provider comum e administrativo; a console usa apenas rotas JRC e o fake é admitido somente no runtime de teste.

JRC-STRENGTH-210

Intenções mutáveis são opacas, voláteis e reutilizadas com segurança

A UI preserva a mesma chave somente para retry incerto e cria outra após nova intenção; leases impedem duplicação concorrente no backend.

JRC-STRENGTH-211

Dependências e browser de teste são fixados no lockfile

CI usa npm ci, Node fixo, Playwright fixado, audit bloqueante para vulnerabilidades HIGH ou CRITICAL e um gate que confere os avisos das dependências distribuídas no browser.

JRC-STRENGTH-212

Contrato e inventário derivam da composição real

O OpenAPI é gerado deterministicamente pela aplicação e cada operação possui política explícita no inventário de segurança.

JRC-STRENGTH-213**Runtime imutável, mínimo e não-root**

A imagem fixa Node por tag e digest, separa build/runtime e executa o entrypoint compilado como usuário node.

Pontos fracos

JRC-SEC-201 · LOW/OPEN**Enforcement final de CSP e Permissions-Policy depende da camada de publicação**

O bundle e a API possuem controles próprios, mas o enforcement dos headers da página HTML depende da camada HTTPS que publicará a console, fora deste incremento sem deploy.

Tabela de achados

ID	Categoria	Severidade	Estado	Arquivo/linhas
JRC-SEC-201	INPUTS_XSS	LOW	OPEN	docs/operations/web-console.md:29-45

Prioridades

- P4 · JRC-SEC-201 — Enforcement final de CSP e Permissions-Policy depende da camada de publicação

Inventário de rotas

GET /v1/api-keys

JWT_OR_JRC_API_KEY · api_keys:manage · RLS SIM

Ownership: RLS_ORGANIZATION_FILTER

Handler: apps/api/src/http/routes/api-keys.ts:133

POST /v1/api-keys

JWT_OR_JRC_API_KEY · api_keys:manage · RLS SIM

Ownership: ACTIVE_ORGANIZATION_CONTEXT

Handler: apps/api/src/http/routes/api-keys.ts:115

DELETE /v1/api-keys/{id}

JWT_OR_JRC_API_KEY · api_keys:manage · RLS SIM

Ownership: RLS_ORGANIZATION_AND_RESOURCE_ID

Handler: apps/api/src/http/routes/api-keys.ts:150

POST /v1/auth/login

NONE · NOT_APPLICABLE_PRE_AUTH · RLS NÃO

Ownership: NOT_APPLICABLE_PRE_AUTH

Handler: apps/api/src/http/routes/auth.ts:148

POST /v1/auth/logout

REFRESH_TOKEN · NOT_APPLICABLE_PRE_AUTH · RLS NÃO

Ownership: REFRESH_TOKEN_FAMILY_BOUND

Handler: apps/api/src/http/routes/auth.ts:246

POST /v1/auth/refresh

REFRESH_TOKEN · NOT_APPLICABLE_PRE_AUTH · RLS NÃO

Ownership: REFRESH_TOKEN_TENANT_BOUND

Handler: apps/api/src/http/routes/auth.ts:223

POST /v1/auth/select-organization

SELECTION_TOKEN · NOT_APPLICABLE_PRE_AUTH · RLS NÃO

Ownership: SELECTION_TOKEN_MEMBERSHIP_REVALIDATED

Handler: apps/api/src/http/routes/auth.ts:192

POST /v1/console/auth/logoutEXACT_ORIGIN_WITH_OPTIONAL_REFRESH_COOKIE_CSRF ·
NOT_APPLICABLE_PRE_AUTH · RLS NÃO

Ownership: REFRESH_TOKEN_FAMILY_WHEN_PRESENT

Handler: apps/api/src/http/routes/console-auth.ts:380

POST /v1/console/auth/restoreREFRESH_COOKIE_CSRF_AND_EXACT_ORIGIN ·
NOT_APPLICABLE_PRE_AUTH · RLS NÃO

Ownership: REFRESH_TOKEN_TENANT_BOUND

Handler: apps/api/src/http/routes/console-auth.ts:336

POST /v1/console/auth/select-organizationSELECTION_TOKEN_AND_EXACT_ORIGIN · NOT_APPLICABLE_PRE_AUTH ·
RLS NÃO

Ownership: SELECTION_TOKEN_MEMBERSHIP_REVALIDATED

Handler: apps/api/src/http/routes/console-auth.ts:313

POST /v1/console/auth/switch-organizationJWT_REFRESH_COOKIE_CSRF_AND_EXACT_ORIGIN ·
ACTIVE_MEMBERSHIP · RLS NÃO

Ownership: SOURCE_AND_TARGET_MEMBERSHIPS_REVALIDATED

Handler: apps/api/src/http/routes/console-auth.ts:353

GET /v1/instances

JWT_OR_JRC_API_KEY · instances:read · RLS SIM

Ownership: RLS_ORGANIZATION_FILTER_AND_TENANT_CURSOR

Handler: apps/api/src/http/routes/instances.ts:149

POST /v1/instances

JWT_OR_JRC_API_KEY · instances:connect · RLS SIM

Ownership: RLS_PROVIDER_ACCOUNT_AND_ORGANIZATION

Handler: apps/api/src/http/routes/instances.ts:123

GET /v1/instances/{id}

JWT_OR_JRC_API_KEY · instances:read · RLS SIM

Ownership: RLS_ORGANIZATION_AND_RESOURCE_ID

Handler: apps/api/src/http/routes/instances.ts:166

POST /v1/instances/{id}/connect

JWT_OR_JRC_API_KEY · instances:connect · RLS SIM

Ownership: RLS_ORGANIZATION_AND_RESOURCE_ID_BEFORE_PROVIDER

Handler: apps/api/src/http/routes/instances.ts:180

POST /v1/instances/{id}/disconnect

JWT_OR_JRC_API_KEY · instances:connect · RLS SIM

Ownership: RLS_ORGANIZATION_AND_RESOURCE_ID_BEFORE_PROVIDER

Handler: apps/api/src/http/routes/instances.ts:226

GET /v1/instances/{id}/status

JWT_OR_JRC_API_KEY · instances:read · RLS SIM

Ownership: RLS_ORGANIZATION_AND_RESOURCE_ID_BEFORE_PROVIDER

Handler: apps/api/src/http/routes/instances.ts:211

GET /v1/provider-accounts

JWT_OR_JRC_API_KEY · instances:read · RLS SIM

Ownership: RLS_ORGANIZATION_FILTER_AND_TENANT_CURSOR

Handler: apps/api/src/http/routes/provider-accounts.ts:71

Varredura sanitizada de segredos

- JRC_GIT_HISTORY: 0 ocorrência(s); escopo reachable root repository blobs; exclusões upstream/evolution-api/**, scripts/security/secret-canaries.mjs.
- COMPILED_BUNDLE: 0 ocorrência(s); escopo apps/api/dist/**, apps/web/dist/**, packages/contracts/dist/**, packages/providers/dist/**, packages/security/dist/**; exclusões **/*.map, node_modules/**, upstream/**.
- VITE_CONFIGURATION: 0 ocorrência(s); escopo process.env names, .env* variable names, apps/web/src production references; exclusões apps/web/src/**/*test.*, apps/web/src/**/*spec.*.

Issues completas

--- ISSUE 1 ---

JRC-SEC-201 — Enforcement final de CSP e Permissions-Policy depende da camada de publicação

Contexto

O Incremento 2 entrega o bundle e a integração local, mas não realiza deploy.

Evidência

`docs/operations/web-console.md:29-45`: política obrigatória da borda HTTPS.

Condições de exploração

Publicação futura sem os headers documentados ou fora da origem da API JRC.

Impacto

Redução da defesa em profundidade contra injeção de conteúdo e recursos desnecessários do navegador.

Correção

Aplicar CSP e Permissions-Policy na borda antes do go-live.

Critérios de aceite

- Console e `/v1` na mesma origem HTTPS.
- CSP sem `unsafe-eval`.
- Permissions-Policy restritiva.
- Teste dos headers no ambiente publicado.

Verificação

Inspecionar os headers e executar o runbook de publicação.

Labels sugeridas

`security`, `frontend`, `deployment`, `hardening`

--- FIM ISSUE 1 ---